

Parallel Logic Expert with Operator Routing

Steven Cleasby-Mayeda
Oregon State University

cleasbys@oregonstate.edu

Abstract

We introduce a logic-augmented transformer pathway that routes token-level representations into fixed fuzzy logical operators (AND/OR/NOT) or straight-through-estimation, so the network learns compositional structure rather than an ensemble of expert networks with shared architecture design. At each backbone layer, the hidden states are projected to a lower dimensional K , Q , which the running logic state queries (cross-attention). The attention matrix is routed, and local windows over either token or gate-channel axes are composed. The final logic state is fused with the backbone features before classification. The proposed architecture augmentation imposes structured, interpretable compositions over token features. A key design dimension is the axis of composition: intra-token compositions (feature axis) compares features within a token, while inter-token compositions (sequence axis) compares features across neighboring tokens. We evaluate these design choices under controlled settings with matched parameter budgets, report task performance, mechanism-level visualizations of the activations of the novel components.

1. Introduction

Transformers provide strong performance on a wide variety of tasks [13], both chain-of-thought (CoT) prompting and reasoning models can greatly improve performance on reasoning tasks [14, 3]. However, many CoT use N-shot prompting. Additionally, both CoT and reasoning models utilize reasoning by iterative, and extended inference, which requires far more tokens. This increases inference costs for many tasks [14]. Our approach instead introduces a novel architecture for structured computation within the latent space, no additional tokens needed. We will test how this constrained, logic-inspired pathway affects performance on reasoning problems and generalization.

Mixture-of-Experts (MoE) models [5, 11, 4] introduce conditional computation by routing tokens to specialized subnetworks, or weighting the results of several/all experts. Our method introduce conditional computation by routing,

weighting tokens to logical operators. In this formulation, the operator routing is learned rather than expert weights. When tied with attention, this method may allow robust routing in the logic space. This space has forced sparsity due to softmax and local compositions (local intra-token or inter-token windows), which may allow emergence of specialized operator-feature connections.

Our activation metrics allows learned logic compositions to be interpreted. Routing entropy and fusion α parameters allow us to measure the utilization of the logic expert. Experiments on logical reasoning benchmarks compare the Baseline, the No-Gate Control network (a baseline with matched parameter count for fair comparison), and the logic expert variants. A core design question is how the local operator windows should be applied in the gate compositions. Our first implementation applies capacity on the channel axis, (neighboring channels on one token). The next design applies capacity on the token axis, each channel compares the same indexed feature on neighboring tokens (channels remain independent). This axis change can have a large impact on which feature relationships are learnable, which has large implications for performance. Figure 1 provides a high-level view of the architecture.

Contributions of this paper are: (1) inter-token compositional routing with operator reductions and residual logic-state updates; (2) controlled ablations over logic dimension, gate count, window size/axis, gate mode, and fusion α ; and (3) evaluation on reasoning and transfer settings with mechanism-level metrics (routing entropy, fusion α , and average gradient norm).

2. Related Work

Logical and logic-inspired neural architectures span a wide variety of designs, from classic weightless systems to modern differentiable operator networks [1]. Some architectures utilize explicit logical, or logic-inspired differentiable operators [9] as an alternative to dense transformations, and they provide evidence that structured logical computation can be more efficient and interpretable than typical transformers. Our approach builds on this motivation, but integrates operator routing directly into trans-

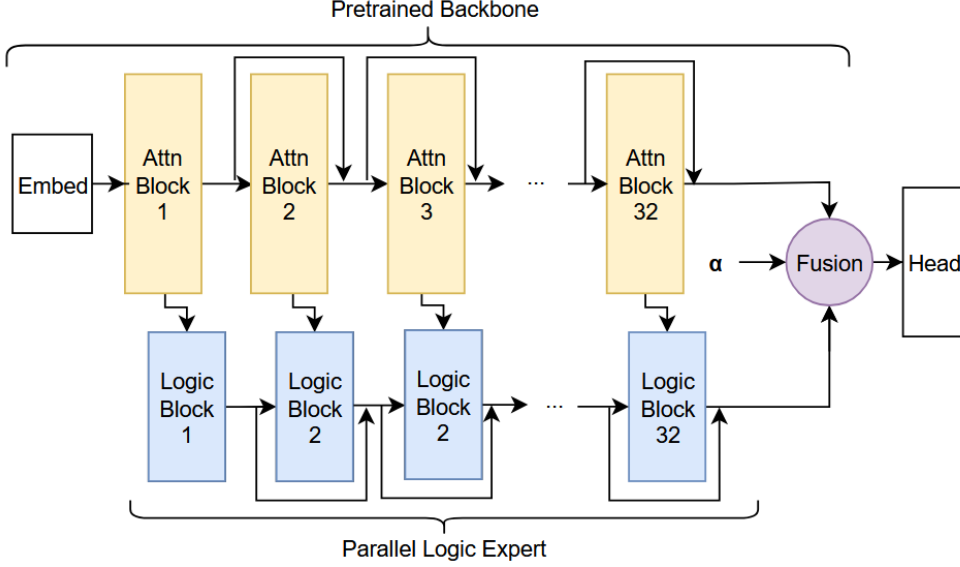


Figure 1. Our architecture, backbone hidden-states feed to logic expert, and fused back with the backbone features before classification

former hidden-state rather than standalone units.

Deep Differentiable Logic Gate Networks combine real-valued logical operators with continuous relaxations (differentiable approximations of discrete logical gates used in gradient descent) and can achieve very efficient inference (one million+ MNIST images/s on a single CPU core) [9]. Logical Neural Networks (LNNs) [10] map neural units to components of weighted logical formulas, yielding highly interpretable representations and supporting logic-consistent reasoning. LNNs use explicitly logic-structured model semantics. Differentiable Logic Machines (DLMs) [17] define a continuous relaxation of first-order logic program spaces (via predicate-weighted parameterization) and learn interpretable logic programs with gradient descent. In contrast to these approaches, which operate over structured relational inputs or operations on the input space, our method integrates logical composition in the feature space, it learns logical compositions over token features.

MoE architectures use a learned gating mechanism to route inputs to specialized expert subnetworks. In adaptive mixtures of local experts, the objective for each expert is decoupled from other experts, enabling specialization over different input regions, and thus allow both specialized and general networks [5]. These models can also improve inference efficiency via forced sparse activations. Conditional computation methods such as MoE and sparse routing motivates learned token-dependent routing decisions [11, 4]. Our model is closest to this line of work, but uses compact differentiable operators instead of expert-token dispatch among independent expert sub-networks.

Fuzzy-set theory provides continuous, differentiable functions [16]. We use fuzzy-set operations to estimate log-

ical operations, which allows training with any off-the-shelf gradient-based optimizer. Our architecture is contrasted by Neural-Symbolic networks, where explicit logic is composed in the non-compressed input space (for example, decision trees or markov-networks) [2].

3. Methods

3.1. Architecture and State Representation

The logic-augmented architecture introduces the logic-expert pathway parallel to the transformer backbone. At each layer t , the backbone produces hidden states:

$$H_t \in \mathbb{R}^{B \times S \times H}. \quad (1)$$

In parallel, the logic expert maintains logic states which are updated at every layer:

$$L_t \in \mathbb{R}^{B \times S \times L}, \quad (2)$$

At each layer, the logic state attends to the backbone via multi-head cross-attention, producing attended logic features:

$$\tilde{L}_t. \quad (3)$$

After the final layer, the logic state is fused with the backbone:

$$H_{\text{fused}} = H_{\text{last}} + \alpha W_f L_{\text{last}}, \quad (4)$$

where α controls the contribution of the logic pathway.

Notation. π_t : routing weights; s_t : gate activations; u_t : routed gate activations; B : batch size; S : sequence length; H : backbone hidden dimension; L : logic-state dimension;

G : gates per operator family (AND/OR/NOT), with total routed channels $3G$; C_t : local window size at layer t ; τ : routing softmax temperature.

3.2. Cross-Attention into Logic Space

At each layer, logic tokens act as queries, while backbone hidden states are projected to keys and values. This produces attended logic features:

$$\tilde{L}_t := \text{CrossAttention}(L_{t-1}, H_t). \quad (5)$$

These features serve as the input to the routing and composition stage.

3.3. Routing and Gate Computation

Routing and gate activation use separate learned projection matrices: W_r produces routing logits, while W_g produces gate activations.

Routings are computed from $\tilde{L}_t$:

$$R_t := \tilde{L}_t W_r, \quad W_r \in \mathbb{R}^{L \times 3G}. \quad (6)$$

Routing weights are:

$$\pi_t[b, s, j] = \frac{\exp(R_t[b, s, j]/\tau)}{\sum_{k=1}^{3G} \exp(R_t[b, s, k]/\tau)}. \quad (7)$$

Gate activations are computed as:

$$s_t := \sigma(\tilde{L}_t W_g), \quad W_g \in \mathbb{R}^{L \times 3G}. \quad (8)$$

The routed soft activations are:

$$u_t^{\text{soft}} := \pi_t \odot s_t. \quad (9)$$

In soft mode, $u_t := u_t^{\text{soft}}$. In STE mode (forward binarized), we use

$$u_t^{\text{hard}} := \mathbf{1}[u_t^{\text{soft}} \geq 0.5], \quad u_t := u_t^{\text{soft}} + \text{sg}(u_t^{\text{hard}} - u_t^{\text{soft}}). \quad (10)$$

We split:

$$[u_t^{\text{AND}}, u_t^{\text{OR}}, u_t^{\text{NOT}}] := \text{split}(u_t; G, G, G). \quad (11)$$

3.4. Windowed Operator Composition

Let $v \in [0, 1]^{B \times S \times G}$ denote one operator group.

Windows are causal and defined along either: inter-token axis (sequence), or intra-token axis (channels).

For inter-token composition:

$$v_{\text{win}} \in \mathbb{R}^{B \times S \times G \times C_t}. \quad (12)$$

Operators are:

$$\text{AND}(x) = \prod_{i=1}^{C_t} x_i, \quad (13)$$

$$\text{OR}(x) = 1 - \prod_{i=1}^{C_t} (1 - x_i), \quad (14)$$

$$\text{NOT}(x) = 1 - x. \quad (15)$$

Applying to routed groups:

$$a_{b,s,g} := \text{AND}(\text{win}(u_t^{\text{AND}})_{b,s,g,:}), \quad (16)$$

$$o_{b,s,g} := \text{OR}(\text{win}(u_t^{\text{OR}})_{b,s,g,:}), \quad (17)$$

$$n_{b,s,g,:} := \text{NOT}(\text{win}(u_t^{\text{NOT}})_{b,s,g,:}). \quad (18)$$

AND and OR reduce to $[B, S, G]$.

NOT is applied elementwise and flattened to $[B, S, G \cdot C_t]$, to preserve negation operation structure.

The concatenated representation has dimension:

$$G + G + G \cdot C_t = G(C_t + 2). \quad (19)$$

3.5. Layer Update and Residual Flow

The concatenated operator outputs are projected:

$$\Delta L_t := \text{Linear}(\text{concat}(a, o, n)). \quad (20)$$

The logic state is updated via:

$$L_t = \text{RMSNorm}(L_{t-1} + \Delta L_t). \quad (21)$$

3.6. Output Fusion

After the final layer, fusion is applied as in Eq. 4. α may be fixed or learned, it directly controls the contribution of the logic expert.

4. Experiments

4.1. Setup

We use Llama-3.1-8B [8] as the shared backbone for all variants. We compare: (i) a Baseline model, (ii) a No-Gate Control (parameter-matched), and (iii+) the logic expert variants. The No-Gate Control serves as a parameter-matched control to separate routing effects from pure parameter-count gains. All variants use the 3 label classifier head. The original Llama head is not used. In this control, the parallel branch excludes cross-attention, operator routing, and compositional gating. All experiments use the AdamW [7] optimizer and cross-entropy loss. Values are collected on a single NVIDIA RTX PRO 6000 Blackwell Server Edition GPU (CUDA 12.8) running on Google Colab.

4.2. Logical Reasoning – ProofWriter

We use the ProofWriter [12] dataset to compare logical reasoning to the baselines. ProofWriter is a logical reasoning dataset where each sample contains a small rulebase (facts plus rules) and a query to evaluate as true, false, or unknown. It is useful here because it directly tests multi-step compositional inference, which matches the intended behavior of the routed logic pathway. We use deterministic, matched sampling across all models (10k train, 5k validation, all problem depths). Unless otherwise noted, all

Model	Validation Accuracy	Validation Loss	Fusion α	Routing Entropy	Avg Grad Norm
Baseline	0.417	1.063	–	–	29.3765
No-Gate Control	0.525	0.820	0.00001 (fixed)	–	11.1387
Routed (intra, $G=8$)	0.542	0.818	0.00001 (fixed)	3.0675	10.5972
Routed (inter, $G=8$)	0.546	0.819	0.1 (fixed)	3.0668	10.4154
Routed (inter, $G=8$)	0.562	0.816	0.01 (fixed)	3.0666	10.4054
Routed (inter, $G=8$)	0.546	0.819	0.006 (learned)	3.0666	10.5527
Routed (inter, $G=16$)	0.501	0.837	0.00001 (fixed)	3.7483	10.9610

Table 1. Validation results. Abbreviations: Baseline = backbone with task classifier head (no parallel branch); No-Gate Control = parameter-matched parallel branch without cross-attention/operator routing/compositional gating; Routed = logic expert variants.

runs use matched seeds. Main runs are trained for 3 epochs, longer training only decreased classification due to overfitting.

4.2.1 Beyond Logic – Multi-Genre Natural Language (MNLI)

To test performance beyond rulebase logic tasks, we evaluate performance on Multi-Genre Natural Language (MNLI) [15]. MNLI allow us to evaluate the natural language understanding of the model. MNLI has logical multi-class labels (Entailment, Neutral, Contradiction), this allows us to map to the same 3-label classification head used for ProofWriter.

4.3. Hyperparameter Evaluations

Primary hyperparameters are logic dimension: L , gate count: G , gate mode: (soft vs STE), gate window size: C_t , window axis: (inter-token vs intra-token), fusion α mode: (fixed vs learned), and initial fusion α . We validate on task validation sets, we use the No-Gate Control architecture to avoid attributing gains to unmatched model size.

4.4. Metrics

Training metrics are validation accuracy and validation loss. Operator mechanism metrics are routing entropy and fusion α . We also report average gradient norm (Avg Grad Norm) as an optimization-health signal. Stable gradient norms generally indicate smoother updates, very large norms can indicate unstable training dynamics, small gradient norms indicates arrested training.

5. Results

5.1. ProofWriter

Table 1 reports final validation accuracy under the evaluation protocol used for each run, validation loss, and fusion parameter state (fixed vs learned) after 3 epochs. The parameter-matched control model substantially outperforms the Baseline, indicating that additional parallel capacity alone already improves performance, while routing

variants provide the strongest overall scores in this setting. The best observed ProofWriter result is the inter-token routed variant with $G = 8$ and fixed $\alpha = 0.01$ (0.562 val accuracy), compared with 0.525 for No-Gate Control and 0.417 for Baseline. Inter-token routing is consistently competitive with or better than intra-token routing at matched gate count, while increasing to $G = 16$ under these settings reduces accuracy (0.501), indicating that more routed channels do not automatically improve quality. Additional hyperparameters listed in the appendix. The mechanism columns in Table 1 show routed variants have substantially lower gradient norms (about 10.41–10.96) than the Baseline (29.38), while No-Gate Control is intermediate (11.14). This is consistent with more stable update magnitudes in the routed settings under the current training regime. Routing entropy is tightly clustered around 3.067 for the $G = 8$ routed runs, but rises to 3.7483 for $G = 16$, where validation quality drops. In this sweep, higher routing entropy at larger gate count appears to coincide with weaker task performance, suggesting a more diffuse routing regime that does not translate into better accuracy.

5.2. Size and Transfer

Model Variant	Trainable Parameters
Baseline	7,504,936,963
No-Gate Control	7,582,192,484
Routed ($G=8$)	7,579,068,164
Routed ($G=16$)	7,579,725,060

Table 2. Trainable parameter counts for variants, inter/intra has no effect

A key confound in models is added trainable parameters. Table 2 confirms close parameter matching between No-Gate Control and routed variants (only small differences), the single-task gains from the best routed model are very unlikely to be explained solely by increased parameter-count.

5.2.1 Multi-Genre Natural Language (MNLI)

MNLI [15]. combines sentence understanding with logic multi-class labels. We map the labels to the ProofWriter classes (important for the next test): contradiction $\rightarrow$ 0, entailment $\rightarrow$ 1, neutral $\rightarrow$ 2. For comparability, evaluation uses matched sampling. Notable hyperparameters values in the appendix.

Model	MNLI Val Acc	MNLI Val Loss
No-Gate Control	0.89	0.34
Routed (intra, $G=8$)	0.88	0.36
Routed (inter, $G=8$)	0.88	0.39

Table 3. Performance on MNLI.

Table 3 reports MNLI results. On MNLI, No-Gate Control achieves the highest validation accuracy (0.888), with routed variants close behind (0.880 intra, 0.878 inter). This suggests broadly similar transfer performance at this scale; without multi-seed variance estimates, these small margins should be interpreted as comparable rather than decisive. Transfer behavior appears dependent on the supervision regime and routing configuration.

5.3. Multi-task Training

For multi-task training, each mini-batch is sampled from a 50/50 mixture of ProofWriter and MNLI. This experiment tests whether routing can jointly optimize symbolic-style and open-domain NLI supervision without degrading either task. Total training steps and optimizer hyperparameters are matched to single-task runs, and we report per-task validation metrics after joint training.

Model	ProofWriter Acc	MNLI Acc	Mean
No-Gate Control	0.544	0.879	0.712
Routed (inter, $G=8$)	0.536	0.881	0.709

Table 4. 50/50 mixed ProofWriter and MNLI samples in the multi-task training experiment.

Table 4 reports the 50/50 multi-task evaluation. In the joint setting, No-Gate Control attains the higher average accuracy (0.712 vs 0.709), while Routed (inter, $G = 8$) attains slightly higher MNLI accuracy (0.881 vs 0.879). This indicates a small task trade-off under 50/50 mixing: routing is competitive in aggregate and can preserve transfer-side performance, but these differences are very small and are best interpreted as similar overall performance under the current multi-task configuration.

6. Discussion

The single-task results show that the routed logic expert variants can outperform both the Baseline and the parameter-matched No-Gate Control, suggesting that gains are not solely due to increased capacity and are influenced

Model	Validation Acc	Validation Loss
Routed-Gate-Only	0.5348	0.832
No-Gate Control	0.4646	1.065

Table 5. ProofWriter evaluation for the earlier version without cross-attention, mean of 3 seeds evaluated.

by compositional routing bias. In contrast, the new 50/50 multi-task results show a tighter regime with near-tied outcomes: No-Gate Control has a slightly higher average accuracy (0.712 vs 0.709), while the routed variant is slightly higher on MNLI within the joint setting (0.881 vs 0.879). Given the tiny margins, these results are best read as similar performance pending multi-seed validation. Single-task MNLI transfer follows the same pattern, while ProofWriter shows larger routed gains. Overall, the evidence suggests a task-dependent benefit profile in which routing is more helpful for logical-composition supervision.

Fusion α provides additional insight: models with moderate nonzero α values indicate that the routing branch is actively contributing to predictions, while near-zero values suggest the model ignores the logic pathway. This highlights the importance of monitoring mechanism-level metrics alongside task performance.

The inter-token capacity formulation changes the inductive bias of gate composition from cross-channel local comparison to cross-token local comparison. This makes each gate channel temporally local while preserving channel independence. Empirically, this design should be interpreted together with routing entropy and fusion α : low entropy with stable nonzero α indicates specialized gates that remain active in final prediction, but these mechanism diagnostics should be interpreted alongside task-level metrics and held-out evaluation [6].

Table 5 suggests that cross-attention is likely not the sole source of gains in the logic expert. However, these values come from an earlier implementation with additional architectural and hyperparameter differences, so both the baseline and logic-expert scores are not directly comparable to the main results in Table 1.

Initialization of the learnable fusion α also affects optimization stability. In our sweeps, values in the 0.01–0.25 range were generally stable, with 0.1 yielding the strongest observed validation accuracy.

6.1. Routing Visualization

Figure 2 visualizes routing specialization directly. We see there the heatmap is sparse, with only a few hot-spots, this implies there are several operators that dominate contribution. This figure is used alongside routing entropy, which summarizes concentration numerically, while the heatmap shows where specialization appears across layers.

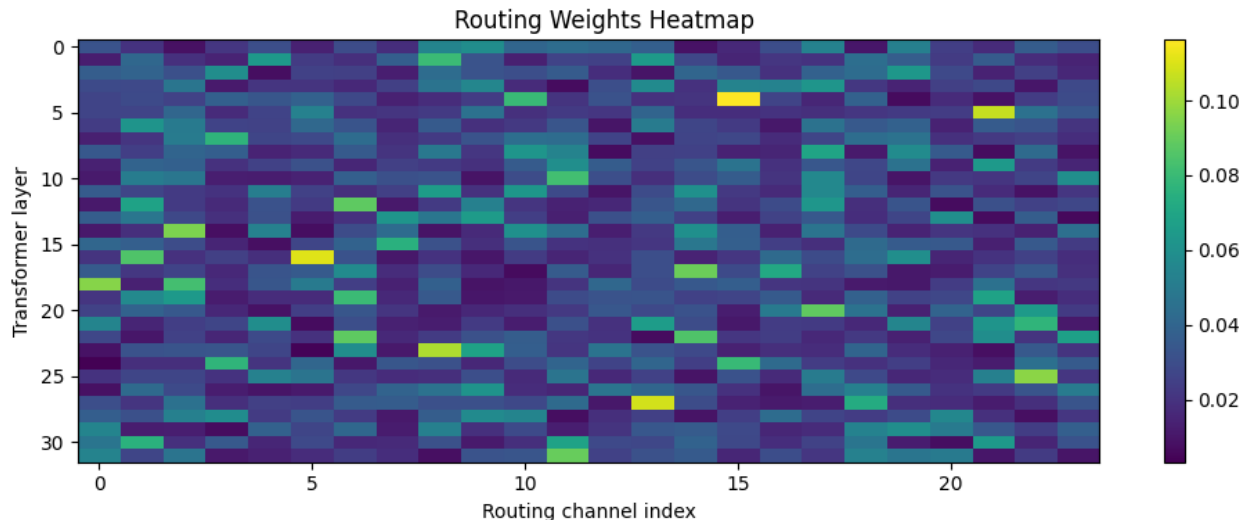


Figure 2. Routing-weight heatmap over transformer depth for a validation batch. Rows are layers and columns are routed channels; color intensity indicates mean routing weight.

7. Limitations

Additional tasks beyond the initial benchmark sets are needed to establish any generalization. We only evaluate one logical reasoning benchmark, more would allow us to evaluate generalization, specifically of the logic expert. The routing design also introduces additional hyperparameters (gate count, window size, routing sparsity) whose interactions are not yet fully explored.

8. Conclusion

This architecture shifts conditional inference from routing among independent neural experts to routing over operator groups, yielding a novel, compact, and interpretable compositional mechanism. Across matched-capacity comparisons, results indicate that the routed logic path can improve over both the backbone Baseline and a parameter-matched No-Gate Control in single-task runs, with inter-token composition emerging as a strong setting in our current sweeps. In MNLI transfer and joint 50/50 multi-task training, routed and No-Gate variants are near-parity under the current setup. Mechanism-level diagnostics (routing entropy and fusion α) provide direct visibility into whether routing is specialized and whether the branch is actually used at prediction time. This work establishes an implementation-complete foundation for operator-routed reasoning. Immediate next steps are broader benchmark coverage, and targeted ablations.

References

- [1] I. Aleksander, M. D. Gregorio, F. M. G. França, P. M. V. Lima, and H. Morton. A brief introduction to weightless neural systems. In *The European Symposium on Artificial Neural Networks*, 2009.
- [2] T. R. Besold, A. d’Avila Garcez, S. Bader, H. Bowman, P. Domingos, P. Hitzler, K.-U. Kuehnberger, L. C. Lamb, D. Lowd, P. M. V. Lima, L. de Penning, G. Pinkas, H. Poon, and G. Zaverucha. Neural-symbolic learning and reasoning: A survey and interpretation, 2017.
- [3] DeepSeek-R1 Team. Deepseek-r1 incentivizes reasoning in llms through reinforcement learning. *Nature*, 645(8081):633–638, Sept. 2025.
- [4] W. Fedus, B. Zoph, and N. Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity, 2022.
- [5] R. A. Jacobs, M. I. Jordan, S. J. Nowlan, and G. E. Hinton. Adaptive mixtures of local experts. *Neural Computation*, 3(1):79–87, 1991.
- [6] D. Keysers, N. Schärli, N. Scales, H. Buisman, D. Furrer, S. Kashubin, N. Momchev, D. Sinopalnikov, L. Stafiniak, T. Tihon, D. Tsarkov, X. Wang, M. van Zee, and O. Bousquet. Measuring compositional generalization: A comprehensive method on realistic data, 2020.
- [7] I. Loshchilov and F. Hutter. Decoupled weight decay regularization, 2019.
- [8] Meta AI. The llama 3 herd of models, 2024.
- [9] F. Petersen, C. Borgelt, H. Kuehne, and O. Deussen. Deep differentiable logic gate networks, 2022.
- [10] R. Riegel, A. Gray, F. Luus, N. Khan, N. Makondo, I. Y. Akhalwaya, H. Qian, R. Fagin, F. Barahona, U. Sharma, S. Ikbali, H. Karanam, S. Neelam, A. Likhyan, and S. Srivastava. Logical neural networks, 2020.
- [11] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, and J. Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer, 2017.

- [12] O. Tafjord, B. D. Mishra, and P. Clark. Proofwriter: Generating implications, proofs, and abductive statements over natural language, 2021.
- [13] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin. Attention is all you need, 2023.
- [14] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, and D. Zhou. Chain-of-thought prompting elicits reasoning in large language models, 2023.
- [15] A. Williams, N. Nangia, and S. Bowman. A broad-coverage challenge corpus for sentence understanding through inference. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers)*, pages 1112–1122. Association for Computational Linguistics, 2018.
- [16] L. Zadeh. Fuzzy sets. *Information and Control*, 8(3):338–353, 1965.
- [17] M. Zimmer, X. Feng, C. Glanois, Z. Jiang, J. Zhang, P. Weng, D. Li, J. Hao, and W. Liu. Differentiable logic machines, 2023.

Appendix

A. ProofWriter Evaluation Details

This appendix summarizes the main ProofWriter configuration used in the experiments.

A.1. Settings and Hyperparameters

- Batch size: 30 (train), 30 (eval)
- Epochs: 3
- Learning rate: 1×10^{-5}
- Weight decay: 0.01

A.2. Dataset(s)

- ProofWriter Depth: `all`: (all allowed)
- Train samples: 10,000
- Validation samples: 5,000

B. MNLI Evaluation Details

For MNLI transfer, labels are remapped into the shared three-class order used throughout this paper: contradiction $\rightarrow$ 0, entailment $\rightarrow$ 1, and neutral $\rightarrow$ 2.

B.1. Dataset(s)

- Train samples: 4,000
- Validation samples: 200

B.2. Settings and Hyperparameters

- Train batch size: 30
- Eval batch size: 100
- Learning rate: 1×10^{-5}
- Weight decay: 0.01
- Logic dimension / gates / heads: 256 / 8 / 4
- Gate mode: `soft`

C. Multi-task Evaluation Details

For multi-task experiments, each batch is sampled from a mix of ProofWriter and MNLI datasets.

C.1. Dataset(s)

- MNLI sampling ratio: 0.5
- ProofWriter sampling ratio: 0.5
- ProofWriter depth: `all`
- Train/validation samples: 10,000 / 1,000

C.2. Settings and Hyperparameters

- Train batch size: 20
- Eval batch size: 100
- Epochs: 6
- Learning rate: 1×10^{-5}
- Weight decay: 0.01
- Logic dimension / gates / heads: 256 / 8 / 4
- Fusion α init: 0.1
- Learn fusion α : `True`
- Gate axis: `inter_token`
- Gate mode: `soft`

D. Routed-Gate-Only vs No-Gate Evaluation Details

This appendix summarizes the overlapping configuration fields for Table 5.

D.1. Settings and Hyperparameters

- Epochs: 6
- Learning rate: 1×10^{-5}

E. ProofWriter Question

This appendix section provides an example question from ProofWriter.

E.1. Rulebase

```
triple1: The cow is big.
triple2: The cow needs the dog.
triple3: The dog sees the rabbit.
triple4: The rabbit chases the cow.
triple5: The rabbit chases the dog.
triple6: The rabbit is big.
triple7: The rabbit sees the dog.
rule1: If the cow is blue and the cow needs the rab
rule2: If the cow chases the dog then the cow sees
rule3: If something is big then it chases the dog.
```

E.2. Question, Answer

```
Question: The cow sees the rabbit?
Answer: true
```